

Guia de Migração do SwagDoc: Swagger 2.0 para OpenAPI 3.2.1

De `svSwagger2` para `svOpenApi3`: o que mudou na especificação, o que alterar no seu código Delphi, quais arquivos entram no deploy e como validar o resultado.

Biblioteca SwagDoc - versão que introduz `TSwagDoc.SpecVersion` (setembro de 2026). Somente Delphi RTL, utilizável em aplicações VCL, FMX, console e servidor.

Sumário

1. Visão geral
2. O modelo de versões
3. O que mudou na especificação
4. Compatibilidade e mudanças incompatíveis
5. Migração passo a passo
6. Migração do código
7. O que um documento Swagger 2.0 não recebe
8. Deploy
9. Validação e testes
10. Limitações conhecidas
11. Checklist de migração
12. Referências

1. Visão geral

O SwaggerDoc agora grava o mesmo modelo de objetos como um documento Swagger 2.0 (`swagger.json`) ou como um documento OpenAPI 3 (`openapi.json`). A família é selecionada pela propriedade `SpecVersion` de `TSwagDoc`.

Pontos principais antes de começar:

- `SpecVersion` tem `svSwagger2` como padrão. Uma aplicação que não usa a nova propriedade continua produzindo o mesmo documento Swagger 2.0 que produzia antes.
- Uma única linha muda a saída para OpenAPI 3.2.1: `vSwagDoc.SpecVersion := svOpenApi3;`
- As classes e propriedades do Swagger 2.0 continuam funcionando quando o documento é gerado como OpenAPI 3. Host, base path, schemes, consumes, produces e os parâmetros body e formData são traduzidos automaticamente.
- Os objetos do OpenAPI 3 (servers, corpos de requisição, media types, links, callbacks, webhooks, requisitos de segurança com escopos, extensões e todos os componentes reutilizáveis) estão disponíveis no modelo de objetos. Eles são traduzidos ou omitidos quando um documento Swagger 2.0 é gerado.
- `LoadFromFile` lê documentos Swagger 2.0 e OpenAPI 3.0, 3.1 e 3.2, de modo que um `swagger.json` existente pode ser convertido.
- Documentos OpenAPI 3.2 precisam do Swagger UI 5. A pasta `Deploy` agora tem uma pasta por família.

Respostas rápidas

Pergunta	Resposta
Preciso alterar meu código depois de atualizar a biblioteca?	Não, a menos que seu código se enquadre em um dos casos de código-fonte da seção 4.3 (enumerações estendidas, definições de segurança personalizadas ou uma lista explícita de units em um package).
A minha saída Swagger 2.0 muda?	Documentos construídos em código mantêm a mesma saída. Documentos carregados de um arquivo <code>swagger.json</code> agora são gravados com mais informações (seção 4.2).
Qual é a menor migração possível?	Definir <code>SpecVersion := svOpenApi3</code> e publicar o <code>openapi.json</code> com os arquivos do Swagger UI 5 de <code>Deploy\OpenApi3</code> .
Posso publicar as duas versões?	Sim. Gere o documento duas vezes a partir do mesmo modelo, alterando <code>SpecVersion</code> (seção 6.15).
Qual versão do OpenAPI é gravada?	A versão 3.x mais recente suportada pelo SwaggerDoc, atualmente a 3.2.1. <code>SwaggerVersion</code> retorna o valor exato.

Estratégias de migração

1. **Apenas trocar.** Mantenha o modelo Swagger 2.0 e defina `SpecVersion := svOpenApi3`. O SwaggerDoc traduz tudo. Recomendado como primeiro passo.
2. **Período de transição.** Publique `swagger.json` e `openapi.json` lado a lado enquanto os consumidores da API migram para o novo documento.
3. **Adoção completa.** Substitua as construções do Swagger 2.0 pelos objetos do OpenAPI 3 (servers, corpos de requisição, content por media type, requisitos de segurança com escopos) e passe a usar os objetos que só existem no OpenAPI 3.

2. O modelo de versões

SpecVersion	Campo raiz do documento	Nome de arquivo padrão	Dialeto de schema
svSwagger2 (padrão)	"swagger": "2.0"	swagger.json	Schema object do Swagger 2.0 (subconjunto do JSON Schema draft 4)
svOpenApi3	"openapi": "3.2.1"	openapi.json	JSON Schema 2020-12

SpecVersion identifica uma **família** da especificação, não uma versão específica:

- Quando uma versão 3.x mais recente passa a ser suportada, `svOpenApi3` começa a gravá-la e o seu código não muda.
- Um novo valor só é adicionado para uma nova família (OpenAPI 4) ou para uma versão incompatível com os documentos já produzidos pela sua família.
- A versão exata gravada no documento é retornada por `SwaggerVersion` ("2.0" ou "3.2.1").

uses

```
Swag.Common.Types, Swag.Doc;
```

```
vSwagDoc.SpecVersion := svOpenApi3;
```

```
vSwagDoc.GenerateSwaggerJson; // vSwagDoc.SwaggerVersion = '3.2.1'
```

```
vSwagDoc.SwaggerFilesFolder := 'C:\MyApi\Help';
```

```
vSwagDoc.SaveSwaggerJsonToFile; // grava C:\MyApi\Help\openapi.json
```

O nome do arquivo segue a família, a menos que `SwaggerFileName` seja atribuído. Atribua `SwaggerFileName := 'swagger.json'` quando a URL publicada não puder mudar.

3. O que mudou na especificação

3.1 Estrutura do documento: Swagger 2.0 comparado com OpenAPI 3.2.1

Área	Swagger 2.0	OpenAPI 3.2.1	Modelo de objetos do SwagDoc
Campo de versão	<code>swagger: "2.0"</code>	<code>openapi: "3.2.1"</code>	<code>SpecVersion</code>
Identidade do documento	nenhuma	<code>\$self</code> , <code>jsonSchemaDialect</code>	<code>SelfUri</code> , <code>JsonSchemaDialect</code>
Servidor de destino	<code>host</code> , <code>basePath</code> , <code>schemes</code>	<code>servers</code> com templates de URL, variáveis e nome	<code>Servers</code> , <code>AddServer</code> , <code>TSwagServer.Name</code>
Media types	<code>consumes</code> , <code>produces</code>	mapa <code>content</code> em cada corpo da requisição, resposta, parâmetro e cabeçalho	<code>AddMediaType</code> de <code>RequestBody</code> , <code>TSwagResponse</code> , <code>TSwagRequestParameter</code> , <code>TSwagHeaders</code>
Payload da requisição	parâmetros <code>body</code> e <code>formData</code>	<code>requestBody</code>	<code>TSwagPathOperation.RequestBody</code>
Objetos reutilizáveis	<code>definitions</code> , <code>parameters</code> , <code>responses</code> , <code>securityDefinitions</code>	<code>components</code> com schemas, responses, parameters, examples, requestBodies, headers, securitySchemes, links, callbacks, pathItems, mediaTypes	<code>Definitions</code> , <code>Responses</code> , <code>Parameters</code> , <code>Examples</code> , <code>RequestBodies</code> , <code>Headers</code> , <code>SecurityDefinitions</code> , <code>Links</code> , <code>Callbacks</code> , <code>PathItems</code> , <code>MediaTypes</code>
Schemas	Schema object, extensão <code>x-nullable</code>	JSON Schema 2020-12 (arrays em <code>type</code> , limites exclusivos numéricos, array <code>examples</code>)	Convertidos automaticamente
Localizações de parâmetros	<code>query</code> , <code>header</code> , <code>path</code> , <code>formData</code> , <code>body</code>	<code>query</code> , <code>header</code> , <code>path</code> , <code>cookie</code> , <code>querystring</code>	<code>rpiCookie</code> , <code>rpiQueryString</code>
Serialização de parâmetros	<code>collectionFormat</code>	<code>style</code> , <code>explode</code> , <code>allowReserved</code> , <code>content</code>	<code>Style</code> , <code>Explode</code> , <code>AllowReserved</code> , <code>Content</code>
Métodos HTTP	<code>get</code> , <code>put</code> , <code>post</code> , <code>delete</code> , <code>options</code> , <code>head</code> , <code>patch</code>	adiciona <code>trace</code> , <code>query</code> e <code>additionalOperations</code>	<code>ohvTrace</code> , <code>ohvQuery</code> , <code>AddAdditionalOperation</code>
Exemplos	<code>examples</code> de uma resposta, por tipo MIME	Example Object com <code>dataValue</code> , <code>serializedValue</code> , <code>externalValue</code>	<code>TSwagExample</code>
Cabeçalhos	<code>type</code> e <code>format</code>	Header Object com schema, content, examples, required, deprecated	<code>TSwagHeaders</code>

Área	Swagger 2.0	OpenAPI 3.2.1	Modelo de objetos do SwagDoc
Relações entre operações	nenhuma	links e callbacks	<code>TSwagLink</code> , <code>TSwagCallback</code>
Eventos enviados pela API	nenhum	webhooks	<code>Webhooks</code>
Esquemas de segurança	basic, apiKey, oauth2 (um fluxo)	http, apiKey (também em cookie), oauth2 (vários fluxos), openIdConnect, mutualTLS	<code>TSwagSecurityDefinitionHttp</code> , <code>...ApiKey</code> , <code>...OAuth2</code> , <code>...OpenIdConnect</code> , <code>...MutualTls</code>
Requisitos de segurança	escopos somente para OAuth2	escopos para todos os esquemas, combinações AND e OR	<code>TSwagSecurityRequirement</code>
Tags	name, description, externalDocs	adiciona summary, parent (tags aninhadas) e kind	<code>Summary</code> , <code>Parent</code> , <code>Kind</code>
Extensões	campos x-	campos x- em todos os objetos	Propriedade <code>Extensions</code>

3.2 Novidades por versão

OpenAPI 3.0

- Servers, components, corpos de requisição, content e media types, encoding de conteúdos multipart e de formulário.
- Callbacks, links, parâmetros de cookie, `style` e `explode`.
- Esquemas de segurança HTTP e OpenID Connect, o Example Object e o Header Object com schema.

OpenAPI 3.1

- Alinhamento completo com o JSON Schema 2020-12: valores nulos são gravados como arrays em `type`, `exclusiveMinimum` e `exclusiveMaximum` são números, e os schemas aceitam um array `examples`.
- Webhooks, `info.summary`, `license.identifier` (SPDX), `components.pathItems`, o esquema de segurança mutual TLS e `jsonSchemaDialect`.
- `summary` e `description` junto a um `$ref` substituem os valores referenciados, e os requisitos de segurança aceitam roles para todos os esquemas.

OpenAPI 3.2

- `$self`, o método QUERY, `additionalOperations` e a localização de parâmetro `querystring`.
- `summary`, `parent` e `kind` de tag, `name` de server e `summary` de resposta.
- `itemSchema` de media type para media types sequenciais (JSON Lines, server-sent events), `prefixEncoding`, `itemEncoding` e `components.mediaTypes`.
- `dataValue` e `serializedValue` de exemplo.
- Fluxo OAuth2 `deviceAuthorization`, `oauth2MetadataUrl` e o campo `deprecated` dos esquemas de segurança.

OpenAPI 3.2.1 é uma versão de correção (patch) da 3.2, com esclarecimentos e correções. Não adiciona novos objetos.

4. Compatibilidade e mudanças incompatíveis

4.1 O que permanece igual

- `TSwagDoc` inicia com `SpecVersion = svSwagger2`, portanto `GenerateSwaggerJson` grava Swagger 2.0 como antes.
- Os documentos construídos em código pelas demos do repositório (`SampleApi`, `GenerateSwaggerJsonFromCode`) são idênticos, byte a byte, à saída da versão anterior. O gerador de Swagger 2.0 só ganhou caminhos de código para propriedades que não existiam antes.
- As units usadas pelas aplicações existentes continuam disponíveis. `Swag.Doc.Path` e `Swag.Doc.Path.Operation.ResponseHeaders` continuam declarando `TSwagPath` e `TSwagHeaders` como aliases das classes que foram movidas (seção 4.3).
- `TJsonSchema` e seus campos produzem os mesmos schemas. A propriedade `Nullable` e o `Format` dos campos string são novos e opcionais.

4.2 Mudanças na saída de documentos carregados de um arquivo swagger.json

Quando um arquivo Swagger 2.0 é carregado com `LoadFromFile` e gerado novamente como Swagger 2.0, a versão anterior perdia parte do documento. São correções, mas o arquivo gerado é diferente:

Conteúdo do arquivo carregado	Versão anterior	Versão atual
<code>securityDefinitions</code>	Não gravado	Gravado
<code>security</code> do documento e das operações	Não gravado, ou gravado sem escopos	Gravado com os escopos (por exemplo <code>["write:pets", "read:pets"]</code>)
<code>externalDocs</code> do documento	Não gravado	Gravado
<code>items</code> e <code>enum</code> de parâmetros array	Perdidos	Gravados
<code>allowEmptyValue</code>	Gravado como <code>true</code> para parâmetros query que não o declaravam	Gravado somente quando declarado
<code>responses</code> do documento e extensões <code>x-</code>	Perdidos	Gravados

Se algum processo posterior compara o arquivo gerado com uma cópia armazenada, atualize a cópia armazenada depois da atualização.

4.3 Mudanças no código-fonte que podem exigir alterações no código

Enumerações estendidas

Enumeração	Valores adicionados
<code>TSwagSecurityDefinitionType</code>	<code>ssdHttp</code> , <code>ssdOpenIdConnect</code> , <code>ssdMutualTls</code>
<code>TSwagRequestParameterInLocation</code>	<code>rpiCookie</code> , <code>rpiQueryString</code>
<code>TSwagPathTypeOperation</code>	<code>ohvQuery</code>

Enumeração	Valores adicionados
TSwagSecurityDefinitionApiKeyInLocation	kilCookie
Novos tipos	TSwagVersion, TSwagRequestParameterStyle, TSwagOAuth2FlowType

Verifique no seu código:

- Arrays constantes indexados por uma dessas enumerações, como `array[TSwagRequestParameterInLocation] of string`. Eles deixam de compilar (E2072, o número de elementos difere) e precisam dos novos valores.
- Loops de `Low` a `High` dessas enumerações. Agora eles percorrem os novos valores; por exemplo, um gerador de código cliente passaria a receber o método `query`.
- Instruções `case` sem ramo `else` que deveriam tratar todos os valores.

Definições de segurança personalizadas

`TSwagSecurityDefinition` ganhou overloads e novos membros virtuais. Uma classe que herda dela continua compilando, mas adicione a diretiva `overload` para que os novos overloads continuem visíveis, e chame `inherited` no seu destructor, pois a classe base agora é dona do objeto `Extensions`.

```
// Versão anterior
TMySecurityDefinition = class(TSwagSecurityDefinition)
protected
    function GetTypeSecurity: TSwagSecurityDefinitionType; override;
public
    function GenerateJsonObject: TJSONObject; override;
    procedure Load(pJson: TJSONObject); override;
end;

// Versão atual
TMySecurityDefinition = class(TSwagSecurityDefinition)
protected
    function GetTypeSecurity: TSwagSecurityDefinitionType; override;
public
    function GenerateJsonObject: TJSONObject; overload; override;
    function GenerateJsonObject(const pVersion: TSwagVersion): TJSONObject; overload; override;
    procedure Load(pJson: TJSONObject); overload; override;
    procedure Load(pJson: TJSONObject; const pVersion: TSwagVersion); overload; override;
    function SupportsVersion(const pVersion: TSwagVersion): Boolean; override;
end;
```

- `GenerateJsonObject(pVersion)` e `Load(pJson, pVersion)` chamam, por padrão, os métodos sem versão; portanto, sobrescreva-os somente quando a representação no OpenAPI 3 for diferente.
- Retorne `False` em `SupportsVersion` para uma família que não consegue representar o esquema. O esquema e os requisitos que o utilizam são então removidos dessa família.
- `Description`, `Deprecated` e `Extensions` estão disponíveis para todos os esquemas.

Classes que foram movidas para outras units

Classe	Unit anterior	Unit atual	A unit anterior ainda funciona?
TSwagPath	Swag.Doc.Path	Swag.Doc.Path.Operation	Sim, Swag.Doc.Path declara um alias
TSwagHeaders	Swag.Doc.Path.Operation.ResponseHeaders	Swag.Doc.Path.Operation.Content	Sim, ResponseHeaders declara um alias

Os aliases são o mesmo tipo, portanto `is`, `as` e as variáveis existentes continuam funcionando.

Novas units

Aplicações que adicionam a pasta `Source` ao library path não precisam de mais nada. Packages ou projetos que listam as units do SwaggerDoc explicitamente devem adicionar:

Unit	Conteúdo
Swag.Common.Json	Funções auxiliares de leitura de JSON
Swag.Doc.Extensions	Extensões da especificação (<code>TSwagExtensions</code>)
Swag.Doc.Example	Example Object (<code>TSwagExample</code>)
Swag.Doc.Link	Link Object (<code>TSwagLink</code>)
Swag.Doc.SecurityRequirement	Requisitos de segurança com escopos
Swag.Doc.Server	Server Object e variáveis de servidor
Swag.Doc.Path.Operation.Content	Media types, encodings e cabeçalhos
Swag.Doc.Path.Operation.RequestBody	Request Body Object
Swag.Doc.SecurityDefinitionHttp	Esquema de segurança HTTP (basic, bearer e outros)
Swag.Doc.SecurityDefinitionOpenIdConnect	Esquema de segurança OpenID Connect
Swag.Doc.SecurityDefinitionMutualTls	Esquema de segurança mutual TLS
Swag.Doc.JsonConverter	Conversão de referências e palavras-chave de schema entre as famílias
Swag.Doc.OpenApi.Generator	Gravador de OpenAPI 3
Swag.Doc.OpenApi.Loader	Leitor de OpenAPI 3

Outros comportamentos importantes

- `LoadFromFile` define `SpecVersion` de acordo com o arquivo carregado. Depois de carregar um arquivo OpenAPI 3, defina `SpecVersion := svSwagger2` se for esperado um documento Swagger 2.0.
- `Extensions.Add` gera `ESwagErrorExtensionName` quando o nome não começa com `x-`.
- Propriedades que recebem valores JSON (`Example`, `DataValue`, `Value`, `Items`, `RequestBody` de um link, `ItemEncoding`) assumem a posse do objeto atribuído. Não o libere.
- `Deploy\index.html`, `Deploy\swagger.json` e os arquivos do Swagger UI da raiz de `Deploy` foram movidos para `Deploy\Swagger2`. Atualize os scripts que os copiam.

5. Migração passo a passo

1. Atualize os fontes do SwaggerDoc e recompile. Se você instala o `Source\SwaggerDoc.dpk`, recompile o package e remova os arquivos `.dcu` e `.bpl` antigos das suas pastas de saída.
2. Corrija os casos de código-fonte da seção 4.3, se houver.
3. Compile a aplicação sem alterar `SpecVersion` e compare o `swagger.json` gerado com o anterior (seção 9.1).
4. Defina `SpecVersion := svOpenApi3`, gere o `openapi.json` e valide-o (seção 9.2).
5. Faça o deploy dos arquivos do Swagger UI 5 de `Deploy\OpenApi3` e configure a URL do documento (seção 8).
6. Teste a página no navegador: operações, schemas, a janela Authorize e as requisições Try it out.
7. Opcionalmente, publique os dois documentos durante um período de transição (seção 6.15).
8. Migre o modelo para os objetos do OpenAPI 3 onde eles melhorarem a documentação (seção 6).
9. Atualize os consumidores da API: geradores de clientes, gateways e testes de contrato que leem o documento.

6. Migração do código

Cada tópico mostra a construção do Swagger 2.0, que continua válida, e a construção do OpenAPI 3. As cláusulas `uses` listam apenas as units necessárias para o exemplo.

6.1 Trocando a versão e mantendo a URL

```
vSwagDoc.SpecVersion := svOpenApi3;
vSwagDoc.GenerateSwaggerJson;
vSwagDoc.SwaggerFilesFolder := 'C:\MyApi\Help';
vSwagDoc.SwaggerFileName := 'swagger.json'; // opcional: mantém o nome de arquivo anterior
vSwagDoc.SaveSwaggerJsonToFile;
```

6.2 Host, base path e schemes se tornam servers

Construção do Swagger 2.0:

```
vSwagDoc.Host := 'api.example.com';
vSwagDoc.BasePath := '/v1';
vSwagDoc.Schemes := [tpsHttps];
```

Quando a lista `Servers` está vazia, o OpenAPI 3 recebe um server por scheme: `https://api.example.com/v1`. Com servers:

```
uses
  Swag.Doc.Server;

var
  vServer: TSwagServer;
  vVariable: TSwagServerVariable;
begin
  vServer := vSwagDoc.AddServer('https://{environment}.example.com/v1', 'Cloud server');
  vServer.Name := 'cloud';
  vVariable := vServer.AddVariable('environment', 'api', 'The environment of the API. ');
  vVariable.Enum.Add('api');
  vVariable.Enum.Add('sandbox');

  vSwagDoc.AddServer('http://localhost:8080/v1', 'Local development server');
end;
```

Os servers não são gravados no Swagger 2.0. Se você também publica o `swagger.json`, mantenha `Host`, `BasePath` e `Schemes`. Quando `Servers` tem itens, o OpenAPI 3 ignora essas três propriedades. `TSwagPath` e `TSwagPathOperation` também têm uma lista `Servers`.

6.3 Parâmetros body se tornam um corpo da requisição

Construção do Swagger 2.0:

```

vParameter := TSwagRequestParameter.Create;
vParameter.Name := 'employee';
vParameter.InLocation := rpiBody;
vParameter.Required := True;
vParameter.Schema.Name := 'Employee';
vOperation.Parameters.Add(vParameter);
vOperation.Consumes.Add('application/json');

```

O OpenAPI 3 recebe um `requestBody` com um media type para cada tipo MIME consumido. Com o objeto de corpo da requisição, cada media type pode ter o seu próprio schema:

```

vOperation.RequestBody.Description := 'The employee data.';
vOperation.RequestBody.Required := True;
vOperation.RequestBody.AddMediaType('application/json').Schema.Name := 'Employee';
vOperation.RequestBody.AddMediaType('application/xml').Schema.Name := 'Employee';

```

No Swagger 2.0, o corpo da requisição é gravado como um parâmetro body com o schema do primeiro media type, e seus media types vão para `consumes`. Isso só acontece quando a operação não tem parâmetro body nem formData.

6.4 Parâmetros formData se tornam um corpo da requisição de formulário ou multipart

Construção do Swagger 2.0:

```

vParameter := TSwagRequestParameter.Create;
vParameter.Name := 'file';
vParameter.InLocation := rpiFormData;
vParameter.TypeParameter := stpFile;
vParameter.Required := True;
vOperation.Parameters.Add(vParameter);

```

O OpenAPI 3 recebe um corpo da requisição `multipart/form-data` com um schema de objeto, e o tipo `file` se torna `type: string, format: binary`. Com o objeto de corpo da requisição:

```

uses
  Swag.Doc.Path.Operation.Content;

var
  vMediaType: TSwagMediaType;

begin
  vMediaType := vOperation.RequestBody.AddMediaType('multipart/form-data');
  vMediaType.Schema.JsonSchema := TJSONObject.ParseJSONValue(
    '{"type":"object","properties":{"title":{"type":"string"},"' +
    '"file":{"type":"string","contentMediaType":"application/pdf"}},' +
    '"required":["file"]}') as
  TJSONObject;
  vMediaType.AddEncoding('file').ContentType := 'application/pdf';
end;

```

No Swagger 2.0, um corpo da requisição de formulário ou multipart é gravado como parâmetros formData. Uma propriedade com `format: binary` ou `contentType` se torna um parâmetro `file`, e uma propriedade `object` se torna um parâmetro `string`.

6.5 Produces e schemas de resposta se tornam content

Construção do Swagger 2.0:

```
vOperation.Produces.Add('application/json');

vResponse := TSwagResponse.Create;
vResponse.StatusCode := '200';
vResponse.Description := 'The employee.';
vResponse.Schema.Name := 'Employee';
vOperation.Responses.Add(vResponse.StatusCode, vResponse);
```

O OpenAPI 3 recebe o schema no `content` da resposta para cada media type produzido (application/json quando nada é produzido). Com content por media type:

```
vResponse.AddMediaType('application/json').Schema.Name := 'Employee';
vResponse.AddMediaType('application/jsonl').ItemSchema.Name := 'Employee'; // um funcionário por linha
```

- Quando `Content` tem itens, `Produces` e `Schema` não são usados para essa resposta no OpenAPI 3.
- No Swagger 2.0, o schema do primeiro media type é gravado quando `Schema` está vazio.
- `Summary` e `Links` de uma resposta são gravados somente no OpenAPI 3.

6.6 Cabeçalhos de resposta

A construção do Swagger 2.0 funciona nas duas famílias:

```
vHeader := vResponse.AddHeader('X-Rate-Limit-Remaining');
vHeader.Description := 'The number of requests left in the current period.';
vHeader.ValueType := 'integer';
```

O Header Object do OpenAPI 3 aceita um schema, exemplos, `Required`, `Deprecated`, `Style`, `Explode` e `Content`:

```
vHeader := vResponse.AddHeader('X-Rate-Limit-Remaining');
vHeader.Description := 'The number of requests left in the current period.';
vHeader.Schema.JsonSchema := TJSONObject.Create.AddPair('type', 'integer');
vHeader.Example := TJSONNumber.Create(99);
```

No Swagger 2.0, `ValueType` e `Format` são obtidos do schema quando estão vazios. Cabeçalhos que são referências não são gravados no Swagger 2.0.

6.7 Exemplos

Construção do Swagger 2.0:

```
vResponse.Examples.Add('application/json', TJsonObject.ParseJSONValue('{ "id":42, "name": "John Smith" }'))
as TJsonObject);
```

O OpenAPI 3 recebe um Example Object por entrada, dentro do content de cada media type produzido. Com o Example Object:

```
uses
    Swag.Doc.Example, Swag.Doc.Path.Operation.Content;

var
    vMediaType: TSwagMediaType;
    vExample: TSwagExample;
begin
    vMediaType := vResponse.AddMediaType('application/json');
    vMediaType.Schema.Name := 'Employee';
    vExample := vMediaType.AddExample('johnSmith');
    vExample.Summary := 'An employee';
    vExample.DataValue := TJsonObject.ParseJSONValue('{ "id":42, "name": "John Smith" }');

    vMediaType := vResponse.AddMediaType('text/csv');
    vMediaType.AddExample('csv').SerializedValue := 'id,name'#10'42,John Smith';
end;
```

- Use apenas um entre `DataValue`, `SerializedValue` e `ExternalValue`. `Value` ainda é aceito, mas não deve ser combinado com `DataValue` ou `SerializedValue`.
- Exemplos reutilizáveis vão para `vSwagDoc.Examples` e são referenciados com `Ref := '#/components/examples/Name'`.
- No Swagger 2.0, o primeiro exemplo de cada media type é gravado em `examples` da resposta.

6.8 Componentes reutilizáveis e referências

As referências escritas para o Swagger 2.0 são reescritas nas duas direções:

Referência no Swagger 2.0	Referência no OpenAPI 3
<code>#/definitions/Name</code>	<code>#/components/schemas/Name</code>
<code>#/parameters/Name</code>	<code>#/components/parameters/Name</code> , ou <code>#/components/requestBodies/Name</code> para parâmetros body e formData
<code>#/responses/Name</code>	<code>#/components/responses/Name</code>

Prefira `Schema.Name` a referências escritas à mão: o SwagDoc grava o caminho correto para cada família. Os componentes que não existem no Swagger 2.0 (examples, headers, links, callbacks, path items e media types) devem ser referenciados com o caminho do OpenAPI 3.

```

vResponse := TSwagResponse.Create;
vResponse.Name := 'notFound'; // chave em components/responses
vResponse.Description := 'The employee was not found.';
vResponse.AddMediaType('application/problem+json').Schema.Name := 'Problem';
vSwagDoc.Responses.Add(vResponse);

vResponse := TSwagResponse.Create;
vResponse.StatusCode := '404';
vResponse.Ref := '#/components/responses/notFound';
vResponse.Summary := 'Unknown employee'; // substitui o summary referenciado
vOperation.Responses.Add(vResponse.StatusCode, vResponse);

```

Propriedade de TSwagDoc	Campo de <code>components</code>	Chave de cada item
Definitions	schemas	Name
Responses	responses	Name
Parameters	parameters	Name
Examples	examples	Name
RequestBodies	requestBodies	Name
Headers	headers	Name
SecurityDefinitions	securitySchemes	SchemeName
Links	links	Name
Callbacks	callbacks	Name
PathItems	pathItems	Uri
MediaTypes	mediaTypes	Name

A `Description` gravada junto com o `Ref` de um parâmetro, corpo da requisição, resposta ou cabeçalho substitui a referenciada no OpenAPI 3, assim como o `Summary` de uma resposta.

6.9 Esquemas e requisitos de segurança

Autenticação basic e bearer

```
// Construção do Swagger 2.0, ainda válida: gravada como http/basic no OpenAPI 3
vBasic := TSwagSecurityDefinitionBasic.Create;
vBasic.SchemeName := 'basicAuth';
vSwagDoc.SecurityDefinitions.Add(vBasic);

// Alternativa do Swagger 2.0 para tokens bearer
vApiKey := TSwagSecurityDefinitionApiKey.Create;
vApiKey.SchemeName := 'bearerAuth';
vApiKey.InLocation := kilHeader;
vApiKey.Name := 'Authorization';
vSwagDoc.SecurityDefinitions.Add(vApiKey);

// OpenAPI 3: esquema HTTP bearer (gravado como a API key acima no Swagger 2.0)
vBearer := TSwagSecurityDefinitionHttp.Create;
vBearer.SchemeName := 'bearerAuth';
vBearer.Scheme := 'bearer';
vBearer.BearerFormat := 'JWT';
vSwagDoc.SecurityDefinitions.Add(vBearer);
```

Fluxos OAuth2

Construção do Swagger 2.0, com um fluxo:

```
vOAuth2 := TSwagSecurityDefinitionOAuth2.Create;
vOAuth2.SchemeName := 'petstore_auth';
vOAuth2.Flow := 'accessCode';
vOAuth2.AuthorizationUrl := 'https://auth.example.com/authorize';
vOAuth2.TokenUrl := 'https://auth.example.com/token';
vOAuth2.AddScope('read:pets', 'Read your pets');
vSwagDoc.SecurityDefinitions.Add(vOAuth2);
```

Os nomes dos fluxos são traduzidos entre as famílias:

Swagger 2.0	OpenAPI 3	TSwagOAuth2FlowType
implicit	implicit	oftImplicit
password	password	oftPassword
application	clientCredentials	oftClientCredentials
accessCode	authorizationCode	oftAuthorizationCode
nenhum	deviceAuthorization	oftDeviceAuthorization

Construção do OpenAPI 3, com vários fluxos:

uses

```
Swag.Doc.SecurityDefinitionOAuth2;
```

var

```
vOAuth2: TSwagSecurityDefinitionOAuth2;  
vFlow: TSwagSecurityDefinitionOAuth2Flow;
```

begin

```
vOAuth2 := TSwagSecurityDefinitionOAuth2.Create;  
vOAuth2.SchemeName := 'oauth2Auth';  
vOAuth2.OAuth2MetadataUrl := 'https://auth.example.com/.well-known/oauth-authorization-server';  
  
vFlow := vOAuth2.AddFlow(oftAuthorizationCode);  
vFlow.AuthorizationUrl := 'https://auth.example.com/authorize';  
vFlow.TokenUrl := 'https://auth.example.com/token';  
vFlow.AddScope('employees:write', 'Creates and updates the employees');  
  
vFlow := vOAuth2.AddFlow(oftClientCredentials);  
vFlow.TokenUrl := 'https://auth.example.com/token';  
vFlow.AddScope('employees:read', 'Reads the employees');  
  
vSwagDoc.SecurityDefinitions.Add(vOAuth2);
```

```
end;
```

No Swagger 2.0, é gravado o primeiro fluxo que existe no Swagger 2.0. Quando `Flows` está vazio, as propriedades de fluxo único são usadas nas duas famílias.

Requisitos de segurança

Construção do Swagger 2.0, sem escopos:

```
vOperation.Security.Add('petstore_auth');
```

Construção do OpenAPI 3, com escopos e combinações:

uses

```
Swag.Doc.SecurityRequirement;
```

var

```
vRequirement: TSwagSecurityRequirement;
```

begin

```
// documento: token bearer OU o escopo de leitura do OAuth2
```

```
vSwagDoc.AddSecurityRequirement.AddScheme('bearerAuth', []);
```

```
vSwagDoc.AddSecurityRequirement.AddScheme('oauth2Auth', ['employees:read']);
```

```
// operação: o escopo de escrita E um certificado de cliente
```

```
vRequirement := vOperation.AddSecurityRequirement;
```

```
vRequirement.AddScheme('oauth2Auth', ['employees:write']);
```

```
vRequirement.AddScheme('mutualTlsAuth', []);
```

```
// operação sem autenticação: security: []
```

```
vHealthOperation.DisableSecurity := True;
```

end;

Cada `AddSecurityRequirement` é uma alternativa (OR) e os esquemas de um requisito são todos obrigatórios (AND). Os requisitos são gravados nas duas famílias. A precedência é:

Nível	Primeira opção	Segunda opção	Terceira opção
Documento	<code>SecurityRequirements</code>	<code>DisableSecurity</code> grava []	<code>GlobalSecurityFromDefinitions</code> grava todas as definições de segurança como alternativas sem escopos
Operação	<code>SecurityRequirements</code>	Lista <code>Security</code> , sem escopos	<code>DisableSecurity</code> grava []

`GlobalSecurityFromDefinitions` é False por padrão, então um documento sem requisitos não grava um `security` próprio e cada operação mantém a segurança que declara. Defina como True para tornar cada esquema uma forma alternativa de chamar a API inteira.

Esquemas em um documento Swagger 2.0

Esquema no modelo	Saída no Swagger 2.0
HTTP basic	<code>type: basic</code>
HTTP bearer ou outro esquema HTTP	<code>type: apiKey, in: header, name: Authorization</code>
OpenID Connect	<code>type: apiKey</code> no cabeçalho <code>Authorization</code> com a extensão <code>x-openIdConnectUrl</code>
API key em um cookie	Removido, junto com os requisitos que o utilizam
Mutual TLS	Removido, junto com os requisitos que o utilizam
OAuth2 apenas com o fluxo device authorization	Removido, junto com os requisitos que o utilizam
<code>Deprecated</code> de qualquer esquema	Não gravado

6.10 Schemas e valores nulos

```
vPhone := vSchema.AddField<string>('phone', 'The employee phone number.');
```

```
vPhone.Nullable := True;
```

As palavras-chave de schema são convertidas para a família que está sendo gerada, inclusive nos JSON schemas atribuídos manualmente a `JsonSchema`:

Construção	Saída no Swagger 2.0	Saída no OpenAPI 3.2.1
Campo <code>Nullable</code> ou <code>nullable: true</code>	Não gravado, ou <code>x-nullable: true</code> quando <code>WriteNullableExtension</code> é True	Array em <code>type</code> que inclui <code>"null"</code>
Referência anulável	Apenas a referência, ou <code>allof</code> com <code>x-nullable</code> quando <code>WriteNullableExtension</code> é True	<code>anyOf</code> com a referência e <code>type: "null"</code>
<code>x-nullable: true</code> lido de um documento Swagger 2.0	Mantido	Array em <code>type</code> que inclui <code>"null"</code>
<code>exclusiveMinimum</code> e <code>exclusiveMaximum</code>	Booleanos, junto com <code>minimum</code> e <code>maximum</code>	Limites numéricos
<code>type: file</code>	Mantido	<code>type: string</code> , <code>format: binary</code>
Array <code>examples</code> do schema	Primeiro item como <code>example</code>	Mantido

Um documento OpenAPI 3.0 carregado com `LoadFromFile` é atualizado quando é gerado novamente.

6.11 Parâmetros

Propriedade	Uso	Saída no Swagger 2.0
<code>InLocation := rpiCookie</code>	Parâmetro enviado em um cookie	Não gravado
<code>InLocation := rpiQueryString</code>	A query string inteira como um único parâmetro, com <code>Content</code> (padrão <code>application/x-www-form-urlencoded</code>)	Não gravado
<code>Style</code> , <code>Explode</code> , <code>AllowReserved</code>	Serialização de arrays e objetos	Não gravado
<code>Deprecated</code>	Parâmetro em processo de descontinuação	Não gravado
<code>Example</code> , <code>Examples</code>	Exemplos do valor	Não gravado
<code>Content</code>	Valores complexos, por exemplo JSON em um parâmetro query	Não gravado
<code>Schema</code>	Qualquer schema, em qualquer localização	Gravado somente para parâmetros body
<code>Description</code> com <code>Ref</code>	Substitui a descrição referenciada	Não gravado

```

vParameter := TSwaggerRequestParameter.Create;
vParameter.Name := 'department';
vParameter.InLocation := rpiQuery;
vParameter.TypeParameter := stpString;
vParameter.Style := rpsForm;
vParameter.Explode := True;
vExample := vParameter.AddExample('sales');
vExample.Summary := 'Sales department';
vExample.DataValue := TJSONString.Create('sales');
vOperation.Parameters.Add(vParameter);

```

6.12 Objetos que existem apenas no OpenAPI 3

Recurso	Código	Saída no Swagger 2.0
URI do documento	<code>vSwagDoc.SelfUri := 'https://api.example.com/v1/openapi.json'</code>	Não gravado
Dialeto de schema	<code>vSwagDoc.JsonSchemaDialect := '...'</code>	Não gravado
Summary do info, licença SPDX	<code>Info.Summary, Info.License.Identifier</code>	Summary não gravado; o identificador se torna a URL da licença quando <code>Url</code> está vazio
Tags aninhadas	<code>Tag.Summary, Tag.Parent, Tag.Kind</code>	Não gravado
Método QUERY	<code>vPath.AddOperation(ohvQuery)</code>	Operação não gravada
Outros métodos	<code>vPath.AddAdditionalOperation('LINK')</code>	Não gravado
Path item reutilizável	<code>vSwagDoc.PathItems</code> e <code>vPath.Ref := '#/components/pathItems/Name'</code>	O path item referenciado é gravado inline
Webhooks	<code>vSwagDoc.Webhooks.Add(vWebhook)</code>	Não gravado
Callbacks	<code>vOperation.AddCallback('Name').AddPathParam('{\$request.body#/callbackUrl}')</code>	Não gravado
Links	<code>vResponse.AddLink('Name')</code>	Não gravado
Media types sequenciais	<code>vMediaType.ItemSchema</code>	Não gravado
Encodings	<code>AddEncoding, AddPrefixEncoding, ItemEncoding</code>	Não gravado
Extensões da especificação	<code>Extensions.Add('x-name', vValue)</code>	Gravadas para os objetos que existem no Swagger 2.0

```

vLink := vResponse.AddLink('GetEmployeeById');
vLink.OperationId := 'getEmployee';
vLink.AddParameter('id', '$response.body#/id');

vCallback := vOperation.AddCallback('exportCompleted');
vCallbackOperation := vCallback.AddPathItem('{ $request.body#/callbackUrl }').AddOperation(ohvPost);
vCallbackOperation.RequestBody.AddMediaType('application/json').Schema.Name := 'ExportResult';

vWebhook := TSwagPath.Create;
vWebhook.Uri := 'employeeHired';
vWebhook.AddOperation(ohvPost).RequestBody.AddMediaType('application/json').Schema.Name := 'Employee';
vSwagDoc.Webhooks.Add(vWebhook);

```

6.13 Convertendo um arquivo swagger.json existente

```

procedure ConvertToOpenApi3(const pSwaggerFile, pOutputFolder: string);
var
    vSwagDoc: TSwagDoc;
begin
    vSwagDoc := TSwagDoc.Create;
    try
        vSwagDoc.LoadFromFile(pSwaggerFile); // SpecVersion passa a ser svSwagger2
        vSwagDoc.SpecVersion := svOpenApi3;
        vSwagDoc.GenerateSwaggerJson;
        vSwagDoc.SwaggerFilesFolder := pOutputFolder;
        vSwagDoc.SaveSwaggerJsonToFile; // openapi.json
    finally
        vSwagDoc.Free;
    end;
end;

```

A direção oposta também funciona. Quando um documento OpenAPI 3 é carregado, `Host`, `BasePath` e `Schemes` são preenchidos a partir do primeiro server, substituindo suas variáveis pelos valores padrão.

6.14 Definições de segurança personalizadas

Consulte a seção 4.3. Um esquema que tem uma representação diferente no OpenAPI 3 sobrescreve `GenerateJsonObject(pVersion)`:

```

function TMySecurityDefinition.GenerateJsonObject(const pVersion: TSwagVersion): TJSONObject;
begin
    if pVersion <> svOpenApi3 then
        Exit(GenerateJsonObject);

    Result := TJSONObject.Create;
    Result.AddPair('type', 'http');
    Result.AddPair('scheme', 'digest');
    if not fDescription.IsEmpty then
        Result.AddPair('description', fDescription);
end;

```

6.15 Publicando as duas famílias a partir do mesmo modelo

```

procedure TApiDocumentation.Publish(const pRootFolder: string);
begin
    fSwagDoc.SpecVersion := svSwagger2;
    fSwagDoc.GenerateSwaggerJson;
    fSwagDoc.SwaggerFilesFolder := TPath.Combine(pRootFolder, 'v2');
    fSwagDoc.SaveSwaggerJsonToFile;           // v2\swagger.json

    fSwagDoc.SpecVersion := svOpenApi3;
    fSwagDoc.GenerateSwaggerJson;
    fSwagDoc.SwaggerFilesFolder := TPath.Combine(pRootFolder, 'v3');
    fSwagDoc.SaveSwaggerJsonToFile;         // v3\openapi.json
end;

```

TPath é declarado em System.IOUtils. Mantenha Host, BasePath e Schemes no modelo para o documento Swagger 2.0 quando usar Servers.

7. O que um documento Swagger 2.0 não recebe

Quando o modelo usa objetos do OpenAPI 3 e o documento é gerado como Swagger 2.0, o SwagDoc grava um documento Swagger 2.0 válido, traduzindo ou omitindo o que o Swagger 2.0 não consegue representar:

Modelo de objetos	Comportamento no Swagger 2.0
RequestBody	Parâmetro body, ou parâmetros formData para conteúdos de formulário e multipart
Content de uma resposta	Schema e primeiro exemplo do primeiro media type
Esquemas HTTP e OpenID Connect	API key no cabeçalho Authorization (seção 6.9)
OAuth2 com vários fluxos	Primeiro fluxo que existe no Swagger 2.0
Requisitos de segurança com esquemas não suportados	Removidos
Parâmetros cookie e querystring	Não gravados
Método QUERY, operações adicionais	Não gravados
Webhooks, callbacks, links	Não gravados
Examples, headers, links, callbacks e media types reutilizáveis	Não gravados
Path items reutilizáveis referenciados por um path	Gravados inline no path
Servers	Não gravados; são usados Host , BasePath e Schemes
SelfUri , JsonSchemaDialect , summary, parent e kind de tag, name de server, summary de resposta	Não gravados
Palavras-chave do JSON Schema 2020-12	Convertidas conforme descrito na seção 6.10

8. Deploy

8.1 Arquivos da biblioteca

O SwaggerDoc depende apenas da Delphi RTL (`System.JSON`, `System.Generics.Collections`, `System.RegularExpressions` e units relacionadas). Ele foi compilado e testado com o Delphi 12 (Studio 23.0). Há duas formas de usá-lo:

- **Library path.** Adicione a pasta `Source` ao library path ou ao search path do projeto. As novas units são encontradas automaticamente.
- **Package de runtime.** Instale o `Source\SwaggerDoc.dpk`. Depois de atualizar, recompile o package e remova os arquivos `.dcu` e `.bpl` antigos, para que as novas units sejam compiladas.

Units da versão atual (N marca as units que são novas nesta versão):

Grupo	Units
Construtor de JSON Schema	<code>Json.Common.Helpers</code> , <code>Json.Schema</code> , <code>Json.Schema.Common.Types</code> , <code>Json.Schema.Field</code> , <code>Json.Schema.Field.Arrays</code> , <code>Json.Schema.Field.Booleans</code> , <code>Json.Schema.Field.Datetimes</code> , <code>Json.Schema.Field.Enums</code> , <code>Json.Schema.Field.Numbers</code> , <code>Json.Schema.Field.Objects</code> , <code>Json.Schema.Field.Strings</code>
Comuns	<code>Swagger.Common.Consts</code> , <code>Swagger.Common.Types</code> , <code>Swagger.Common.Types.Helpers</code> , <code>Swagger.Common.Json</code> (N)
Documento	<code>Swagger.Doc</code> , <code>Swagger.Doc.Info</code> , <code>Swagger.Doc.Info.Contact</code> , <code>Swagger.Doc.Info.License</code> , <code>Swagger.Doc.Tags</code> , <code>Swagger.Doc.Definition</code> , <code>Swagger.Doc.Server</code> (N), <code>Swagger.Doc.Extensions</code> (N), <code>Swagger.Doc.Example</code> (N), <code>Swagger.Doc.Link</code> (N)
Paths e operações	<code>Swagger.Doc.Path</code> (alias), <code>Swagger.Doc.Path.Operation</code> , <code>Swagger.Doc.Path.Operation.RequestParameter</code> , <code>Swagger.Doc.Path.Operation.RequestBody</code> (N), <code>Swagger.Doc.Path.Operation.Response</code> , <code>Swagger.Doc.Path.Operation.ResponseHeaders</code> (alias), <code>Swagger.Doc.Path.Operation.Content</code> (N)
Segurança	<code>Swagger.Doc.SecurityDefinition</code> , <code>Swagger.Doc.SecurityDefinitionBasic</code> , <code>Swagger.Doc.SecurityDefinitionApiKey</code> , <code>Swagger.Doc.SecurityDefinitionOAuth2</code> , <code>Swagger.Doc.SecurityDefinitionHttp</code> (N), <code>Swagger.Doc.SecurityDefinitionOpenIdConnect</code> (N), <code>Swagger.Doc.SecurityDefinitionMutualTls</code> (N), <code>Swagger.Doc.SecurityRequirement</code> (N)
Leitura, gravação e conversão	<code>Swagger.Doc.FileLoader</code> , <code>Swagger.Doc.JsonConverter</code> (N), <code>Swagger.Doc.OpenApi.Generator</code> (N), <code>Swagger.Doc.OpenApi.Loader</code> (N)

As classes de definição de segurança se registram na sua seção `initialization`. `LoadFromFile` só as encontra quando suas units estão linkadas, o que `Swagger.Doc.FileLoader` já garante. Mantenha essas units no projeto quando as units do SwaggerDoc forem listadas explicitamente.

8.2 Arquivos do Swagger UI

A pasta `Deploy` tem uma pasta por família. Copie a pasta da família que você publica para o servidor web e adicione o documento gerado.

Deploy\Swagger2 - Swagger UI 3.3.1, para swagger.json

Arquivo	Obrigatório	Função
index.html	Sim	Página e configuração. A opção <code>url</code> é <code>/api/help/swagger.json</code> ; altere-a para o endereço do seu documento
swagger-ui.css	Sim	Estilos
swagger-ui-bundle.js	Sim	Swagger UI
swagger-ui-standalone-preset.js	Sim	Layout da barra superior
oauth2-redirect.html	Para OAuth2	Recebe o token da janela Authorize
favicon-16x16.png , favicon-32x32.png	Não	Ícones
swagger-ui.js	Não	Versão em módulo do Swagger UI, não usada pelo <code>index.html</code>
*.map	Não	Source maps, usados apenas para depurar o Swagger UI
swagger.json	Sim	Gerado pelo SwagDoc com <code>svSwagger2</code>
readme.txt	Não	Instruções

A página também carrega fontes de `fonts.googleapis.com`. Sem acesso à internet, a página usa fontes locais.

Deploy\OpenApi3 - Swagger UI 5.32.15, para openapi.json

Arquivo	Obrigatório	Função
index.html	Sim	Página
index.css	Sim	Estilos da página
swagger-initializer.js	Sim	Configuração. A opção <code>url</code> é <code>./openapi.json</code>
swagger-ui.css	Sim	Estilos
swagger-ui-bundle.js	Sim	Swagger UI
swagger-ui-standalone-preset.js	Sim	Layout da barra superior
oauth2-redirect.html	Para OAuth2	Recebe o token da janela Authorize
favicon-16x16.png , favicon-32x32.png	Não	Ícones
LICENSE , NOTICE , *.LICENSE.txt	Ao redistribuir	Avisos da Apache License 2.0 do Swagger UI
openapi.json	Sim	Gerado pelo SwagDoc com <code>svOpenApi3</code>
readme.txt	Não	Instruções

O Swagger UI 3.x não consegue renderizar documentos OpenAPI 3.1 ou 3.2. Publique o `openapi.json` com o Swagger UI 5 (os arquivos de `Deploy\OpenApi3`, ou uma versão 5.x mais recente). O Swagger UI 5 também renderiza documentos Swagger 2.0.

8.3 Configurando a página

`swagger-initializer.js` de `Deploy\OpenApi3`:

```

window.onload = function() {
  window.ui = SwaggerUIBundle({
    url: "/api/help/openapi.json",          // endereço do documento no seu servidor
    validatorUrl: null,
    dom_id: '#swagger-ui',
    deepLinking: true,
    presets: [SwaggerUIBundle.presets.apis, SwaggerUIStandalonePreset],
    plugins: [SwaggerUIBundle.plugins.DownloadUrl],
    layout: "StandaloneLayout"
  });
};

```

Para oferecer os dois documentos em uma única página do Swagger UI 5, substitua `url` por `urls`:

```

urls: [
  { url: "/api/help/openapi.json", name: "OpenAPI 3.2.1" },
  { url: "/api/help/swagger.json", name: "Swagger 2.0" }
],
"urls.primaryName": "OpenAPI 3.2.1",

```

8.4 Checklist do servidor web

- Sirva a página via HTTP ou HTTPS. Os navegadores bloqueiam a requisição que carrega o documento quando o `index.html` é aberto a partir do sistema de arquivos.
- Sirva os arquivos `.json` com `Content-Type: application/json` e UTF-8.
- Quando o documento for servido por uma origem diferente da página, envie os cabeçalhos CORS (`Access-Control-Allow-Origin`).
- Gere o documento novamente quando a API mudar (na inicialização ou no build) e evite caches desatualizados: envie `Cache-Control: no-cache` para o documento ou adicione uma versão à sua URL.
- Para OAuth2, registre o endereço do `oauth2-redirect.html` como redirect URI no servidor de autorização.
- Proteja a rota da documentação quando a API não for pública.

8.5 Estrutura de transição

```

/api/help/v2/   arquivos de Deploy\Swagger2 + swagger.json   (consumidores existentes)
/api/help/v3/   arquivos de Deploy\OpenApi3 + openapi.json   (novos consumidores)

```

Mantenha as duas até que os consumidores (geradores de clientes, API gateways, testes de contrato) leiam o documento OpenAPI 3 e, então, remova a rota v2.

8.6 Mudanças na estrutura do repositório

Versão anterior	Versão atual
Deploy\index.html, Deploy\swagger.json e os arquivos do Swagger UI na raiz de Deploy	Deploy\Swagger2

Versão anterior	Versão atual
A demo <code>SampleApi</code> grava o seu executável e o <code>swagger.json</code> em <code>Deploy</code>	Grava-os em <code>Deploy\Swagger2</code>
nenhuma	<code>Deploy\OpenApi3</code> com Swagger UI 5.32.15 e <code>openapi.json</code>
nenhuma	<code>Demos\SampleOpenApi3</code> : a versão OpenAPI 3.2.1 da <code>SampleApi</code> , gravando em <code>Deploy\OpenApi3</code>

8.7 Publicando o documento pela própria aplicação

Uma API escrita com um framework web publica o documento pelo próprio servidor, em vez de copiar um arquivo para um servidor web. A pasta `Integrations` do repositório tem uma página por framework.

Com o [Horse](#), o middleware de `Integrations\Horse` publica o documento e a página que o apresenta. Uma aplicação que já o utiliza muda uma propriedade:

```
THorse.Use(HorseSwagDoc);

SwagDocApi.SpecVersion := svOpenApi3;
SwagDocConfig.DocumentRoute := '/docs/openapi.json';
```

Com o [DelphiMVCFramework](#), o `SwagDoc` é distribuído dentro da pasta `lib/swagdoc` do framework e o middleware `MVCFramework.Middleware.Swagger` monta o documento a cada requisição. Publicar um documento OpenAPI 3 exige uma versão distribuída que declare `TSwagVersion` e o middleware definindo `SpecVersion := svOpenApi3`.

A página que apresenta o documento também precisa entender a família. Os arquivos do Swagger UI publicados por uma aplicação escrita para o Swagger 2.0 são substituídos pela distribuição de `Deploy\OpenApi3`.

9. Validação e testes

9.1 Regressão do Swagger 2.0

Gere o `swagger.json` com a versão anterior e com a versão atual, sem alterar `SpecVersion`, e compare os arquivos:

```
fc /b before\swagger.json after\swagger.json
```

Documentos construídos em código devem ser idênticos. Documentos carregados de arquivos podem ser diferentes, conforme descrito na seção 4.2.

9.2 Validação do OpenAPI 3.2.1

- Valide o `openapi.json` com o JSON Schema da especificação OpenAPI 3.2 publicado em <https://spec.openapis.org>, usando qualquer validador de JSON Schema 2020-12.
- Abra o documento no Swagger UI 5 e verifique as operações, os schemas, a janela Authorize e as requisições Try it out.
- Carregue o arquivo gerado com `LoadFromFile` e gere-o novamente: o resultado deve ser idêntico ao arquivo original.

9.3 Validação do Swagger 2.0

- Valide o `swagger.json` no Swagger Editor ou com um validador de Swagger 2.0.
- O Delphi grava a barra das strings JSON como `\`. Isso é JSON válido, mas ferramentas que interpretam o texto como YAML (o Swagger Editor 4, por exemplo) informam "unknown escape character". Reformate o JSON antes de colá-lo nessas ferramentas. A versão anterior tinha a mesma saída.

10. Limitações conhecidas

- O Swagger UI 5.32.15 não lista os webhooks de documentos OpenAPI 3.2 (somente de documentos 3.1) e não exibe `additionalOperations`. Ambos estão presentes no `openapi.json`.
- O Swagger UI 5.32.15 exibe um aviso quando `jsonSchemaDialect` é diferente de `https://spec.openapis.org/oas/3.1/dialect/base`. Deixe `JsonSchemaDialect` vazio, a menos que os schemas usem outro dialeto.
- A conversão para Swagger 2.0 omite os objetos listados na seção 7.
- `Servers` não são convertidos para `host`, `basePath` e `schemes` quando um documento construído em código é gerado como Swagger 2.0.
- Uma classe de definição de segurança personalizada não pode ser lida por `LoadFromFile`, porque os tipos de esquema são uma enumeração fechada.

11. Checklist de migração

- ☐ Fontes do SwagDoc atualizados, package recompilado e arquivos `.dcu` e `.bp1` antigos removidos.
- ☐ Arrays, loops e instruções `case` sobre as enumerações estendidas revisados.
- ☐ Definições de segurança personalizadas atualizadas com `overload` e `SupportsVersion`.
- ☐ Novas units adicionadas aos packages que listam as units do SwagDoc.
- ☐ Saída Swagger 2.0 comparada com a da versão anterior.
- ☐ `SpecVersion := svOpenApi3` definido, e `SwaggerFileName` atribuído se a URL não puder mudar.
- ☐ `openapi.json` validado com o schema do OpenAPI 3.2.
- ☐ Arquivos do Swagger UI 5 de `Deploy\OpenApi3` publicados e `swagger-initializer.js` apontando para o documento.
- ☐ Servidor web servindo JSON com o content type correto e com os cabeçalhos de CORS e de cache.
- ☐ Redirect URI do OAuth2 registrada para `oauth2-redirect.html`.
- ☐ Scripts atualizados para as pastas `Deploy\Swagger2` e `Deploy\OpenApi3`.
- ☐ `Host`, `BasePath` e `Schemes` mantidos se o `swagger.json` ainda for publicado.
- ☐ Requisitos de segurança declarados explicitamente, com escopos.
- ☐ Consumidores da API informados e migrados para o documento OpenAPI 3.

12. Referências

- Especificação Swagger 2.0: <https://github.com/OAI/OpenAPI-Specification/blob/main/versions/2.0.md>
- Especificação OpenAPI 3.2.1: <https://github.com/OAI/OpenAPI-Specification/blob/main/versions/3.2.1.md>
- JSON Schema 2020-12: <https://json-schema.org/draft/2020-12>
- Distribuição do Swagger UI: <https://github.com/swagger-api/swagger-ui/tree/master/dist>
- Repositório do SwagDoc: <https://github.com/marcelojaloto/SwagDoc>
- Demos do SwagDoc: `Demos\SampleApi` (Swagger 2.0) e `Demos\SampleOpenApi3` (OpenAPI 3.2.1)
- Aplicações de exemplo que publicam um documento OpenAPI 3.2.1:
<https://github.com/marcelojaloto/Delphi/tree/master/samples> — `server-api-rest-dmvc`
(DelphiMVCFramework) e `tasks-manager-horse` (Horse)